MBLP131 · Hafta 9 · Ders Notları

İşletim Sistemleri Temelleri

Linux Kabuğuna Giriş ve Dosya Sisteminde Gezinme

Kuramdan terminale: neredeyim, burada ne var ve nasıl hareket ederim?

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. Bir laboratuvar bilgisayarında, Windows'ta WSL ile ya da bir VirtualBox sanal makinesinde Linux terminali açmak ve dersin laboratuvar klasörü `~/lab`'ı oluşturmak.
2. Terminal, kabuk ve çekirdek arasındaki farkı açıklamak; bir komut yazdığınızda neler olduğunu (fork, exec, wait, sistem çağrıları) anlatmak.
3. İstemi okumak ve bir komut satırını komut, seçenekler ve argümanlar olarak ayırmak.
4. `whoami`, `hostname`, `date`, `echo`, `uname`, `pwd` ve en kullanışlı seçenekleriyle `ls` komutlarını kullanmak; `ls -l` çıktısının her sütununu okumak.
5. Linux dizin ağacını (`/`, `/home`, `/etc`, `/bin`, `/usr`, `/var`, `/tmp`, `/dev`, `/proc`) tanımlamak.
6. `cd` ile mutlak ve göreli yollar ve `~` `.` `..` `-` özel adlarıyla dolaşmak; `cd`'nin neden bir kabuk yerleşiği olduğunu açıklamak.
7. Yardım bulmak (`--help`, `man`, `help`, `type`); sekme tamamlama, komut geçmişi ve klavye kısayollarıyla verimli çalışmak.

İçindekiler

1 Vizeden Terminale 1

2 Linux Terminaline Ulaşmak 2

3 Terminal, Kabuk ve Çekirdek 4

4 İstem ve İlk Komutlar 6

5 Neredeyim? Burada Ne Var? 7

6 Linux Dizin Ağacı 11

7 Dolaşmak: cd ve Yollar 13

8 Yardım Almak 15

9 Akıllı Çalışma: Tamamlama, Geçmiş, Kısayollar 16

10 Komut Başvurusu: 9. Hafta 18

11 Sık Görülen Hatalar ve Anlamları 18

12 Özet 19

13 Sözlük 20

14 Laboratuvar Föyü 20

15 Tekrar Soruları 22

16 Laboratuvar Föyü Çözümleri 23

17 İleriye Bakış: 10. Hafta 25

1 Vizeden Terminale

Dersin ilk yarısı (1–7. haftalar; 8. haftadaki vizede sınandı) *fikirlerle* ilgiliydi: kaynak yöneticisi ve genişletilmiş makine olarak işletim sistemi; çekirdek modunda çalışan çekirdek ve kullanıcı modunda çalışan geri kalan her şey; çekirdeğe açılan tek kapı olarak **sistem çağrıları** (system calls; 2–3. haftalar); PID'si, durumu ve PCB'siyle, `fork()` ve `exec()` ile oluşturulan ve işlemci zamanlayıcısı tarafından sıralanan **süreçler** (processes; 3–5. haftalar); ve son olarak bellek, sanal bellek, dosyalar ve ağaç yapılı dizinler (6–7. haftalar).

Bu haftadan 14. haftaya kadar Linux kabuğunda *uygulamalı* çalışıyoruz. Aynı fikirleri iş başında göreceksiniz: yazdığınız her komut bir sürece dönüşür; her dosya listesi bir dizinden bir sistem çağrısıyla okunur; 7. haftanın dizin ağacı, `cd` ile içinde yürüdüğünüz ağaçtır. 10. hafta dosya işlemlerini, 11. hafta izinleri, 12. hafta metin işleme ve boruları (pipes), 13. hafta süreçleri ve iş denetimini, 14. hafta kabuk betiklerini (shell scripting) işleyecek.

Bu notlardaki çıktılar hakkında. Bütün terminal çıktıları Ubuntu 24.04 LTS üzerinde, `student` adlı normal bir kullanıcıyla, `okan-lab` adlı bir bilgisayarda ve `LC_ALL =C.UTF-8` yerel ayarıyla (locale) alındı. Sizin makinenizde kullanıcı adı, bilgisayar adı, tarihler, saatler, sistem dosyalarının boyutları ve süreç numaraları (PID'ler) farklı olacaktır; başka bir yerel ayarla `ls`'nin sıralaması da biraz değişebilir (örneğin büyük harfle başlayan adların önce gelip gelmemesi). `~/lab` içindeki dosyalar ise herkes için aynıdır. Ubuntu ayrıca `ls` çıktısını renklendirir (dizinler mavi); bizim kaydettiğimiz çıktıda renk yoktur. Her gösterim `code/` klasöründe bir `demo_*.sh` dosyası olarak, çıktısıyla birlikte `out_*.txt` dosyasında durur.

Türkçe sistemler. Türkçe yerel ayarlı bir sistemde (`LANG =tr_TR.UTF-8`) birçok program mesajlarını Türkçe yazar: örneğin `No such file or directory` yerine `Böyle bir dosya ya da dizin yok`, `Permission denied` yerine `Erişim engellendi` görebilirsiniz; `date` Türkçe gün ve ay adları kullanır, `ls`'nin sıralaması da farklı olabilir. Bu notlardaki çıktılar `LC_ALL =C.UTF-8` ile, yani İngilizce mesajlarla alınmıştır; İngilizce mesajlar internette arama yaparken de daha çok sonuç verir. Bir komutu tek seferlik İngilizce çalıştırmak için başına `LC_ALL =C.UTF-8` yazabilirsiniz: `LC_ALL =C.UTF-8 ls nowhere`.

2 Linux Terminaline Ulaşmak

Dönemin geri kalanı için bir Linux kabuğuna ihtiyacınız var. Ona ulaşmanın üç iyi yolu vardır; komutlar üçünde de aynıdır.

2.1 1. seçenek: laboratuvar bilgisayarları

Oturum açın, *Terminal* uygulamasını başlatın ve Bölüm 2.4 ile devam edin. *[Laboratuvar bilgisayarlarındaki derslik, hesap ve sistem öğretim üyesi tarafından duyurulacaktır.]*

2.2 2. seçenek: Windows 10/11'de WSL

Linux için Windows Alt Sistemi (Windows Subsystem for Linux, WSL), Windows'un yönettiği hafif bir sanal makinede gerçek bir Linux çekirdeği çalıştırır; varsayılan dağıtım Ubuntu'dur.

1. Başlat düğmesine sağ tıklayın ve *Terminal (Yönetici)* ya da *Windows PowerShell (Yönetici)* seçeneğini seçin.
2. Aşağıdaki komutu yazıp Enter'a basın:

   ```
   PS C:\Windows\system32> wsl --install
   ```

   Komut gerekli Windows özelliklerini etkinleştirir, WSL 2'yi kurar ve Ubuntu'yu indirir.
3. Windows isteyince bilgisayarı yeniden başlatın.
4. Başlat menüsünden *Ubuntu*'yu açın. İlk açılışta yeni bir *Linux* kullanıcı adı ve parolası ister (Windows hesabınızla aynı olmaları gerekmez). Parolayı yazarken ekranda *hiçbir şey görünmez*, yıldız bile — Linux'ta bu normaldir. Bu kullanıcı daha sonra `sudo` ile yönetici komutları çalıştırabilir.
5. Bundan sonra kabuğa ihtiyaç duyduğunuzda *Ubuntu*'yu açın (ya da PowerShell'de `wsl` yazın).

Yararlı bilgiler: `wsl -l -v` (PowerShell'de) kurulu dağıtımları ve WSL sürümlerini listeler; `wsl --install -d Ubuntu-24.04` belirli bir Ubuntu sürümünü kurar. Linux içinde Windows `C:` sürücünüz `/mnt/c` olarak görünür; dolayısıyla Windows'taki indirme klasörünüz `/mnt/c/Users/<Windows adınız>/Downloads`'tır. Kurulum sanallaştırmayla ilgili bir mesajla başarısız olursa bilgisayarınızın BIOS/UEFI ayarlarında sanallaştırmayı açın (Intel VT-x, AMD-V ya da SVM adıyla geçer).

2.3 3. seçenek: VirtualBox ve Ubuntu

Sanal makine (virtual machine), işletim sistemindeki sanallaştırma fikrini bütün bir bilgisayara uygular: bir *hipervizör* (burada VirtualBox) konuk işletim sistemine sanal işlemciler, bellek ve sanal bir disk verir; Ubuntu donanımın sahibiymiş gibi çalışır.

1. VirtualBox'ı indirip kurun ve *Ubuntu Desktop 24.04 LTS* ISO kalıbını indirin (yaklaşık 6 GB).
2. VirtualBox'ta *New* (Yeni) düğmesine tıklayın: makineye bir ad verin, ISO kalıbını seçin, tür olarak Linux / Ubuntu (64-bit).
3. Makineye 4096 MB bellek, 2 işlemci ve 25 GB'lık bir sanal disk verin (dinamik olarak ayrılan: dosya yalnızca gerektikçe büyür). Dizüstü bilgisayarınızda yalnızca 4 GB RAM varsa WSL'yi tercih edin.
4. Sanal makineyi başlatın ve *Install Ubuntu* adımlarını izleyin. "Erase disk and install Ubuntu" seçeneği yalnızca *sanal* diski, yani bilgisayarınızdaki bir dosyayı siler — Windows diskinizi değil. (VirtualBox 7, kullanıcı bilgilerini sizin yerinize dolduran bir "gözetimsiz kurulum" da yapabilir.)
5. Oturum açın ve `Ctrl+Alt+T` ile (ya da *Etkinlikler* → *Terminal*) bir terminal açın.

Mac kullananlar. macOS Terminali (`zsh` kabuğuyla) bu haftanın komutlarının çoğunu anlar, ancak macOS'taki BSD araçlarının birçok seçeneği Linux'taki GNU araçlarından farklıdır. Önümüzdeki haftalar için bir Linux sanal makinesi kullanın (Apple Silicon Mac'lerde UTM ücretsiz bir seçenektir; VirtualBox desteği orada sınırlıdır).

2.4 Laboratuvar klasörünü oluşturmak

Bütün uygulamalı haftalar aynı alıştırma klasörünü, `~/lab`'ı kullanır. Bu klasörü `mcmp117_lab_setup.sh` betiği (script) oluşturur; betiği her Linux haftasının `code/` klasöründe ve dersin Telegram kanalında bulursunuz *[indirme bağlantısı: öğretim üyesi tarafından teyit edilecek]*. Betiği `bash` ile *bir kez* çalıştırın; betiğin kendisi nerede olursa olsun, `lab` klasörünü her zaman ev dizininizde kurar:

```
# Ubuntu / sanal makine: betik indirme klasörünüzde
student@okan-lab:~$ bash ~/Downloads/mcmp117_lab_setup.sh
# WSL: betik Windows'taki indirme klasörünüzde (Windows adınızı yazın)
student@okan-lab:~$ bash /mnt/c/Users/ayse/Downloads/mcmp117_lab_setup.sh
```

Betik ev dizinindeyken gerçek çıktı şöyledir (`ls` bir dizini listeler; Bölüm 5):

```
student@okan-lab:~$ cd
student@okan-lab:~$ pwd
/home/student
student@okan-lab:~$ ls
lab  mcmp117_lab_setup.sh
student@okan-lab:~$ bash mcmp117_lab_setup.sh
Lab folder ready: /home/student/lab
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~$ ls
lab  mcmp117_lab_setup.sh
student@okan-lab:~$ ls lab
README.txt  backup  data  docs  logs  photos  projects  scripts  src
```

Betikte rastgelelik yoktur: herkes bu notlardakiyle tıpatıp aynı dosyaları elde eder. Yeniden çalıştırmak güvenlidir — *yalnızca* `lab` klasörünü silip yeniden oluşturur — bu yüzden lab klasörünüz karıştığında betiği yeniden çalıştırmanız yeter. Aynı nedenle kendi çalışmalarınızı asla `~/lab` içinde tutmayın.

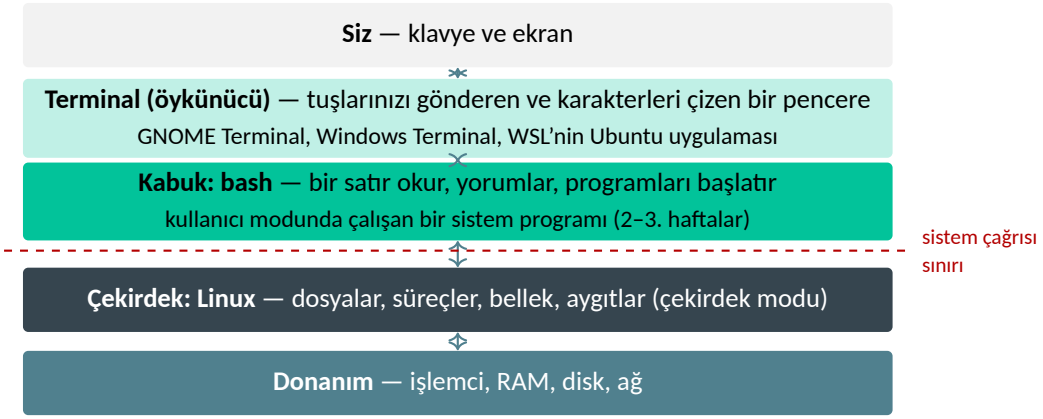
Betik hata verirse

`line 2: $'\r': command not found` gibi bir hata görürseniz dosya Windows satır sonlarını almıştır (örneğin Not Defteri'nde açılıp kaydedildikten sonra). Dosyayı yeniden indirin ya da `sed -i 's/\r$//' mcmp117_lab_setup.sh` ile onarın. `No such file or directory` görürseniz önce yolu `ls` ile kontrol edin (Bölüm 11).

3 Terminal, Kabuk ve Çekirdek

3.1 Üç program, üç görev

İnsanlar "terminal", "kabuk", "komut satırı" ve "Linux" sözcüklerini çoğu zaman aynı şeyi kastediyormuş gibi kullanır. Oysa bunlar farklı katmanlardır:

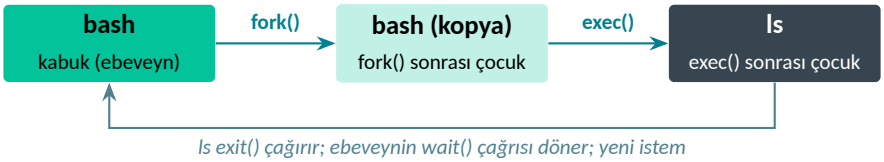


Şekil 1: Sizinle donanım arasındaki katmanlar. Kesikli çizginin üstündeki her şey kullanıcı modunda çalışır.

- **Terminal** yalnızca girdi ve çıktıyla ilgilenir; komutları anlamaz. (Tarihsel olarak terminal fiziksel bir aygıttı — bir teleks makinesi ya da VT100 gibi bir video terminali; bugün onu taklit eden, yani ona *öykünen* bir programdır.)
- **Kabuk** (shell), 3. haftadaki *komut yorumlayıcısıdır* (command interpreter). Linux birkaç kabuk sunar: `bash` (Ubuntu'da varsayılan), `zsh`, `sh`, `fish`. 2. haftada vurguladığımız gibi kabuk çekirdeğin bir parçası *değildir*; sıradan bir sistem programıdır.
- **Çekirdek** (kernel), her komut için asıl işi yapar ve ona yalnızca sistem çağrılarıyla ulaşılır.
- Tam anlamıyla "Linux" yalnızca çekirdektir; Ubuntu bir *dağıtımdır* (distribution): Linux çekirdeği artı GNU araçları (`bash`, `ls`, …) ve binlerce başka program.

3.2 Bir komut yazdığınızda ne olur?

`ls` yazıp Enter'a bastığınızda `bash`, 3. haftadaki *fork–exec–wait* kalıbını izler:



Şekil 2: Kabuk her harici komut için yeni bir süreç oluşturur.

1. **Oku:** `bash` terminalden `ls` satırını okur.
2. **Bul:** `ls` bir yerleşik değildir; bu yüzden `bash`, `$PATH` değişkeninde listelenen dizinleri arar ve `/usr/bin/ls` program dosyasını bulur.
3. **fork():** `bash` yeni bir PID'si olan bir çocuk süreç, yani kendinin bir kopyasını oluşturur.
4. **exec():** çocuk kendi programını `/usr/bin/ls` ile değiştirir; `ls` çalışır, yazar ve sonlanır.
5. **wait():** ebeveyn kabuk çocuğun bitmesini bekler, sonra yeni istemi yazar.

Yerleşik (builtin) denen bazı komutları ise `bash` yeni bir süreç oluşturmadan kendisi yürütür: `cd`, `pwd`, `echo`, `history`, `type` ve diğerleri. Bölüm 7.4, `cd`'nin neden yerleşik *olmak zorunda* olduğunu açıklıyor.

3.3 Komutlar sistem çağrısına dönüşür

2. haftada bir C programının sistem çağrılarını listelemek için `strace` kullanmıştık. Araç her komutla çalışır. Burada yalnızca üç tür sistem çağrısını izliyoruz; `> /dev/null` ise `ls`'nin normal çıktısını atıyor, böylece yalnızca `strace`'in raporu kalıyor (yönlendirme — redirection — 12. haftanın konusu):

```
student@okan-lab:~/lab$ strace -e trace=execve,getdents64,write ls docs > /dev/null
execve("/usr/bin/ls", ["ls", "docs"], 0x7ffc5b53bce8 /* 13 vars */) = 0
getdents64(3, 0x55fd38daa8e0 /* 9 entries */, 32768) = 288
getdents64(3, 0x55fd38daa8e0 /* 0 entries */, 32768) = 0
write(1, "final report.txt\nlinux_quotes.tx"..., 86) = 86
+++ exited with 0 +++
```

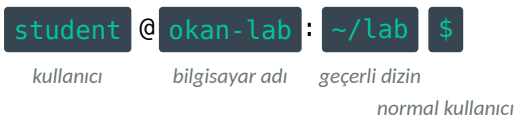
- `execve`: çocuk süreç, `["ls", "docs"]` argüman listesiyle `/usr/bin/ls`'ye dönüşür — yukarıdaki 4. adım.
- `getdents64` ("get directory entries", dizin girdilerini al) dizini okur. 9 girdi buldu: 6 görünür dosya, gizli `.settings` dosyası ve her dizinin içerdiği `.` ve `..` girdileri. İkinci çağrı 0 girdi döndürür: dizinin sonu. 7. haftadan hatırlayın: dizin, (ad, düğüm numarası) çiftlerini içeren bir dosyadır; bu çiftleri okuyan sistem çağrısı budur.
- `write(1, …, 86)`, 1 numaralı dosya tanımlayıcısına (file descriptor), yani standart çıktıya 86 bayt yazar.

`ls` diski kendi başına okuyamaz; kullanıcı modundaki her program gibi çekirdekten istemek zorundadır (ikili mod, 2. hafta). Süzgeç olmadan `ls` birkaç düzine sistem çağrısı yapar (kitaplıkları yükleme, yerel ayar dosyalarını okuma, bellek eşleme). `strace` kurulu değilse: `sudo apt install strace`.

4 İstem ve İlk Komutlar

4.1 İstemi okumak

Kabuk bir komuta hazır olduğunda bir **istem** (prompt) yazar. Ubuntu'nun varsayılan istemi `kullanıcı@bilgisayar:dizin$` biçimindedir:



Tilde işareti ~, **ev dizininizin** (home directory) kısaltmasıdır; burada `/home/student`. `$` yerine `#` ile biten bir istem, yönetici olan `root` kullanıcısı olduğunuzu gösterir — o zaman dikkatli olun. Bu notlarda istemden sonraki metin sizin yazdığınız şeydir; istemsiz satırlar çıktıdır.

4.2 Bir komutun anatomisi

Bir komut satırı boşluklardan *sözcüklere* bölünür:

komut `-seçenekler argümanlar` örneğin `ls -l -a docs`

- İlk sözcük **komuttur**: bir program dosyası ya da bir yerleşik.
- **Seçenekler** (options) komutun *nasıl* çalışacağını değiştirir. Kısa seçenekler bir tireden sonra tek harftir ve birleştirilebilir: `ls -l -a` = `ls -la` = `ls -al`. Uzun seçenekler iki tireyle başlar ve açık yazılır: `ls --all`. Bazı seçenekler bir değer alır: `head -n 4`.
- **Argümanlar** (arguments) komutun *ne* üzerinde çalışacağını söyler: dosyalar, dizinler, metin.

Linux *büyük-küçük harfe duyarlıdır*: `ls` ile `LS` farklı sözcüklerdir, `docs` ile `Docs` da öyle. Boşluklar da önemlidir: `ls -l` tek bir sözcüktür, yani var olmayan bir komut.

Örnek	Komut	Seçenekler	Argümanlar
`pwd`	`pwd`	—	—
`ls -la docs`	`ls`	`-l -a` (birleşik)	`docs`
`ls --all docs`	`ls`	`--all` (uzun biçim)	`docs`
`cd ../src`	`cd`	—	`../src`
`head -n 4 file`	`head`	`-n`, değeri `4`	`file`

4.3 İlk komutlar

```
student@okan-lab:~$ whoami
student
student@okan-lab:~$ hostname
okan-lab
student@okan-lab:~$ date
Mon Oct 12 08:49:13 UTC 2026
student@okan-lab:~$ echo Hello from the shell
Hello from the shell
student@okan-lab:~$ echo My shell is $SHELL and my home is ~
My shell is /bin/bash and my home is /home/student
```

```
student@okan-lab:~$ uname
Linux
student@okan-lab:~$ uname -r
6.18.44-fc-v114
student@okan-lab:~$ uname -a
Linux okan-lab 6.18.44-fc-v114 #1 SMP PREEMPT_DYNAMIC @0 x86_64 x86_64 x86_64 GNU/Linux
```

- `whoami` kullanıcı adınızı, `hostname` bilgisayarın adını, `date` tarihi ve saati yazar (test makinemizde UTC; sizinki yerel saati gösterir).
- `echo` argümanlarını yazar. Dikkat edin: `echo` hiçbir zaman `$SHELL` ya da `~` görmedi; komut başlamadan önce *kabuk* onların yerine `/bin/bash` ve `/home/student` koydu. Kabuğun yaptığı bu *genişletme* (expansion), 10. hafta (joker karakterler) ve 14. hafta (değişkenler) için temel bir fikirdir.
- `uname` çekirdeğin adını yazar; `-r` çekirdek sürümünü; `-a` her şeyi (çekirdek adı, bilgisayar adı, sürüm, derleme bilgisi, makine mimarisi `x86_64`). Normal bir Ubuntu 24.04 bilgisayarda sürüm `6.8.0-…-generic` gibi görünür; WSL'de `microsoft-standard-WSL2` ile biter.
- `clear` (ya da `Ctrl+L`) ekranı temizler.

5 Neredeyim? Burada Ne Var?

Her sürecin — dolayısıyla her kabuğun — bir **geçerli çalışma dizini** (current working directory) vardır. Göreli yollar (Bölüm 7) buradan başlar. `pwd` (*print working directory*, çalışma dizinini yaz) onu gösterir; `ls` (*list*, listele) içinde ne olduğunu gösterir.

5.1 pwd ve ls

```
student@okan-lab:~$ cd ~/lab
student@okan-lab:~/lab$ pwd
/home/student/lab
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~/lab$ ls -F
README.txt  backup/  data/  docs/  logs/  photos/  projects/  scripts/  src/
student@okan-lab:~/lab$ ls docs
'final report.txt'   notes_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ ls -a docs
 .    .settings          linux_quotes.txt   notes_02.txt   syllabus.txt
 ..  'final report.txt'   notes_01.txt       notes_03.txt
```

- `pwd` her zaman kök `/` ile başlayan *mutlak* bir yol yazar.
- Argümansız `ls` geçerli dizini listeler; `ls docs` başka bir dizini listeler. Adlar sıralanır ve sütunlara dizilir.
- `-F` türü belirten bir işaret ekler: dizinler için `/` (ayrıca çalıştırılabilir dosyalar için `*`, sembolik bağlantılar için `@`).
- `-a` (*all*, tümü) **gizli** adları da gösterir; bunlar yalnızca nokta ile başlayan adlardır: burada `.settings`. Yapılandırma dosyaları genellikle gizlidir, örneğin `~/.bashrc`. Ayrıca `.` (dizinin kendisi) ve `..` (üst

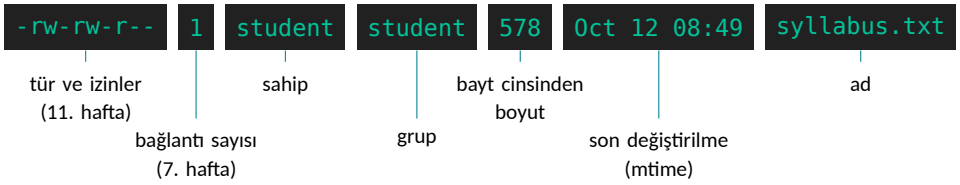
dizini) girdilerini de görürsünüz — tam olarak 7. haftadaki özel girdiler. `-A` gizli dosyaları `.` ve `..` olmadan gösterir.

- `'final report.txt'` adı tırnak içinde gösterilir, çünkü bir boşluk içerir. Tırnaklar adın parçası *değildir*; GNU `ls` onları adın nerede başlayıp bittiğini görebilmeniz için ekler.

5.2 Uzun liste: ls -l

```
student@okan-lab:~/lab$ ls -l docs
total 24
-rw-rw-r-- 1 student student  53 Oct 12 08:49 'final report.txt'
-rw-rw-r-- 1 student student 302 Oct 12 08:49  linux_quotes.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_01.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_02.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_03.txt
-rw-rw-r-- 1 student student 578 Oct 12 08:49  syllabus.txt
```

`ls -l` çıktısının her satırı bir dosyayı tanımlar. Sütunlar şunlardır:



Şekil 3: `ls -l` çıktısındaki bir satırın yedi sütunu.

1. **Tür ve izinler.** İlk karakter türdür: `-` normal dosya, `d` dizin, `l` sembolik bağlantı, `c` karakter aygıtı. Sonraki dokuz karakter sahip, grup ve diğer herkes için okuma/yazma/çalıştırma izinleridir (permissions); 11. hafta tamamen bunlara ayrılmıştır.
2. **Bağlantı sayısı** (link count). Bir dosya için: dosyanın düğümünü (inode) gösteren sabit bağlantıların (adların) sayısı (7. hafta). Bir dizin için: $2 +$ alt dizinlerinin sayısı (aşağıya bakın).
3. Dosyanın **sahibi** ve **grubu**.
4. Bayt cinsinden **boyut**. Bir dizin kendi girdi tablosunun boyutunu gösterir; bu genellikle 4096 baytlık tek bir bloktur.
5. **Değiştirilme zamanı** (mtime): *içeriğin* son değişimi. Altı aydan eski dosyalarda saat yerine yıl görünür (Bölüm 6'deki `Apr 22 2024`).
6. **Ad.**

ls -l düğümü gösterir

Ad dışında `ls -l` çıktısının her sütunu dosyanın *düğümünün* (inode, 7. hafta) bir alanıdır: tür, izinler, bağlantı sayısı, sahip, grup, boyut ve zamanlar. Ad ise dizin girdisinden gelir. 10. hafta `stat` ile düğümün tamamını gösterecek.

Dizinlerin bağlantı sayıları. `projects` dizininin bağlantı sayısı neden 4, `docs` dizininki neden yalnızca 2? Bir dizine üst dizindeki girdisi, kendi `.` girdisi ve her alt dizinindeki `..` girdisi ad verir. Dolayısıyla

$$\text{bir dizinin bağlantı sayısı} = 2 + s,$$

burada s alt dizin sayısıdır: `projects` için $s = 2$ (`alpha` ve `beta`), yani $2 + 2 = 4$; `docs` için $s = 0$, yani 2. (`code/check_numbers.py` ile denetlendi.)

`total` satırı. İlk satır, `total 24`, listelenen dosyaların 1 KiB birimleriyle kullandığı disk alanıdır. 16 baytlık bir dosya bile dosya sisteminde koca bir 4 KiB'lık blok kaplar; bu yüzden altı küçük dosya $6 \times 4 = 24$ KiB kullanır (7. haftadaki blok fikri). `~/lab` dizininin kendisinde `ls -l` `total 36` gösterir: dokuz girdi, $9 \times 4 = 36$.

5.3 İnsan tarafından okunabilir boyutlar: ls -lh

```
student@okan-lab:~/lab$ ls -lh logs data
data:
total 12K
-rw-rw-r-- 1 student student  227 Oct 12 08:49 cities.txt
-rw-rw-r-- 1 student student   80 Oct 12 08:49 numbers.txt
-rw-rw-r-- 1 student student 1.2K Oct 12 08:49 students.csv

logs:
total 32K
-rw-rw-r-- 1 student student  24K Oct 12 08:49 access.log
```

`-h` seçeneğiyle boyutlar K, M, G ekleriyle yazılır; bunlar burada 1024'ün kuvvetleri demektir: $1\,\text{K} = 2^{10} = 1024$ bayt, $1\,\text{M} = 2^{20}$ bayt. `ls` *yukarı* yuvarlar ve 10'dan küçük değerlerde bir ondalık basamak gösterir:

Çözümlü örnek: bayttan ls -h'ye

$$\begin{aligned} \texttt{access.log}: &\quad 23\,604/1024 \approx 23.05 \longrightarrow \texttt{24K}\ (\text{yukarı yuvarlanır}) \\ \texttt{system.log}: &\quad 5\,207/1024 \approx 5.085 \longrightarrow \texttt{5.1K} \\ \texttt{students.csv}: &\quad 1\,208/1024 \approx 1.180 \longrightarrow \texttt{1.2K} \end{aligned}$$

1024 bayttan küçük dosyalar (örneğin 227 baytlık `cities.txt`) bayt olarak gösterilir. Tam bayt sayıları `ls -l` çıktısından gelir (Laboratuvar Föyü, B7 alıştırması). Programın kendisi ondalık nokta kullanır (`5.1K`); Türkçe yerel ayarlı bir sistemde `5,1K` görebilirsiniz.

5.4 Bütün alt ağaçlar: ls -R ve tree

`ls -R` (*recursive*, özyinelemeli) bir dizini ve ardından altındaki her alt dizini listeler; her dizin için bir blok. `tree` programı aynı bilgiyi resim olarak çizer — 7. haftanın ağacı. (`tree` kurulmayı gerektirebilir: `sudo apt install tree`.)

```
student@okan-lab:~/lab$ ls -R projects
projects:
alpha  beta

projects/alpha:
README.md  docs  src

projects/alpha/docs:
design.txt

projects/alpha/src:
main.c
```

… (`projects/beta` için de aynısı; çıktının tamamı `code/out_tree.txt` içinde)

```
student@okan-lab:~/lab$ tree projects
projects
├── alpha
│   ├── README.md
│   ├── docs
│   │   └── design.txt
│   └── src
│       └── main.c
└── beta
    ├── README.md
    ├── docs
    │   └── design.txt
    └── src
        └── main.c

7 directories, 6 files
```

`tree projects` dizininin kendisini de sayar; “7 directories” bundandır. **Dikkat:** `ls -R /` bütün dosya sistemini listeler — yüz binlerce satır. Çok uzun süren bir komutu durdurmak için `Ctrl+C`’ye basın.

5.5 En kullanışlı ls seçenekleri

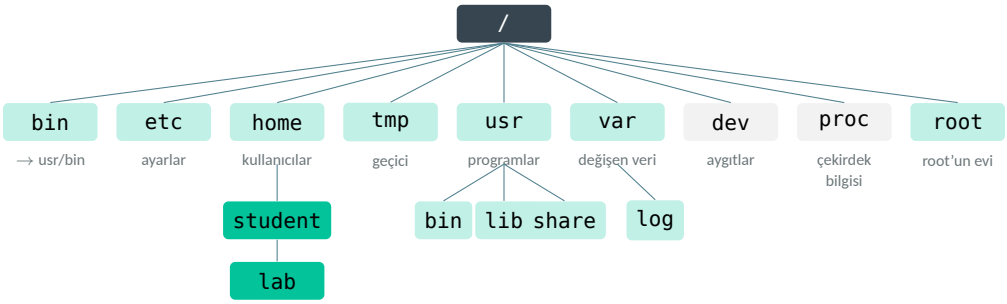
Seçenek	Anlamı	Örnek
`-l`	uzun liste (Şekil 3)	`ls -l docs`
`-a / -A`	gizliler dâhil bütün adlar (-A: `.` ve `..` olmadan)	`ls -a ~`
`-h`	insan tarafından okunabilir boyutlar (-l ile)	`ls -lh logs`
`-R`	özyinelemeli: her alt dizini de	`ls -R projects`
`-F`	türü işaretle: `/` dizin, `*` çalıştırılabilir, `@` bağlantı	`ls -F`
`-d`	içeriği değil, dizinin kendisi	`ls -ld projects`
`-S`	boyuta göre sırala, en büyük önce	`ls -lS docs`
`-t`	değiştirilme zamanına göre sırala, en yeni önce	`ls -lt`
`-r`	sıralamayı ters çevir	`ls -ltr`
`-1`	her satırda bir ad	`ls -1 photos`
`-i`	düğüm numaralarını göster (7. hafta)	`ls -i`

Ubuntu’nun varsayılan `~/.bashrc` dosyası ayrıca şu kısayolları (*takma adlar*, aliases) tanımlar: `ll` = `ls`

`-alF`, `la` = `ls -A` ve `l` = `ls -CF`.

6 Linux Dizin Ağacı

Linux'ta *tek* bir ağaç ve *tek* bir kök, `/`, vardır. Sürücü harfleri yoktur: başka diskler ve USB bellekler ağaca *bağlanır* (mount; genellikle `/media` ya da `/mnt` altına). **Dosya Sistemi Hiyerarşi Standardı** (Filesystem Hierarchy Standard, FHS) hangi dizinin neyi tuttuğunu belirler; böylece bütün dağıtımlar birbirine benzer.



Şekil 4: Linux dizin ağacının üst kısmı. Gri: çekirdeğin ürettiği sanal dosya sistemleri.

Dizin	İçerik	Deneyin
`/`	bütün ağacın kökü	`ls /`
`/home`	her kullanıcı için bir ev dizini: `/home/student`	`ls ~`
`/root`	yönetici `root`'un ev dizini (`/` ile aynı şey değildir)	`---`
`/etc`	sistem genelindeki yapılandırma, çoğunlukla düz metin dosyaları	`head -n 4 /etc/os-release`
`/usr`	kurulu yazılımlar: `/usr/bin` programlar, `/usr/lib` kitaplıklar, `/usr/share` veriler	`ls -l /usr/bin/ls`
`/bin, /sbin, /lib`	Ubuntu 24.04'te `/usr/bin`, `/usr/sbin`, `/usr/lib`'e sembolik bağlantılar	`ls -ld /bin`
`/var`	büyüyen değişken veriler: günlükler (`/var/log`), önbellekler, posta, kuyruklar	`ls /var`
`/tmp`	geçici dosyalar; herkes yazabilir; açılışta sıkça boşaltılır	`ls -ld /tmp`
`/dev`	aygıt dosyaları: diskler, terminaller, `/dev/null`	`ls -l /dev/null`
`/proc`	sanal: süreçler ve çekirdek hakkında anında üretilen bilgi	`head /proc/meminfo`
`/media, /mnt`	çıkarılabilir ve diğer diskler için bağlama noktaları	`---`
`/boot`	çekirdek kalıbı ve önyükleyici dosyaları	`---`

6.1 Bir tur

Aşağıdaki `/` listesinden yalnızca test makinemizde bulunan iki girdi çıkarıldı. `.usr-is-merged` ile biten dizinler Ubuntu'nun "birleşik `/usr`" yapısına geçişinden kalan zararsız artıklardır.

```
student@okan-lab:~$ type ls
ls is /usr/bin/ls
student@okan-lab:~$ cd /
student@okan-lab:/$ ls
bin                etc                lib64       opt   sbin                tmp
bin.usr-is-merged  home               lost+found  proc  sbin.usr-is-merged  usr
boot               lib                media       root  srv                 var
dev                lib.usr-is-merged  mnt         run   sys
student@okan-lab:/$ ls -ld /bin /sbin /lib
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /lib -> usr/lib
lrwxrwxrwx 1 root root 8 Apr 22  2024 /sbin -> usr/sbin
student@okan-lab:/$ ls -l /usr/bin/ls
-rwxr-xr-x 1 root root 142312 Jan 23  2026 /usr/bin/ls
student@okan-lab:/$ head -n 4 /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
student@okan-lab:/$ ls /var
backups  cache  lib  local  lock  log  mail  opt  run  spool  tmp
student@okan-lab:/$ ls -ld /tmp
drwxrwxrwt 16 root root 4096 Oct 12 08:49 /tmp
```

- `type ls`, `ls`'nin sıradan bir program dosyası olduğunu gösterir: `/usr/bin/ls` ($142\,312$ bayt). Komutların çoğu bunun gibi dosyalardır.
- `/bin`, `/sbin` ve `/lib`, `/usr` içindeki karşılıklarına giden **sembolik bağlantılardır** (symbolic links; tür `l`, ok `->`; 7. hafta). İki ad da aynı programlara götürür.
- `/etc/os-release` dağıtımın adını verir. UNIX yapılandırmayı düzenlenebilir metin olarak tutar; 11. hafta `/etc/passwd` ve `/etc/group` dosyalarını okuyacak.
- `/tmp`'nin izin dizgesi `drwxrwxrwt`'dir: orada herkes dosya oluşturabilir; sondaki `t` (*yapışkan bit*, sticky bit, 11. hafta) ise kullanıcıların birbirlerinin dosyalarını silmesini engeller.

6.2 /dev ve /proc: her şey bir dosyadır

```
student@okan-lab:/$ ls -l /dev/null /dev/tty /dev/zero
crw-rw-rw- 1 root root 1, 3 Oct  9 23:24 /dev/null
crw-rw-rw- 1 root root 5, 0 Oct  9 23:24 /dev/tty
crw-rw-rw- 1 root root 1, 5 Oct  9 23:24 /dev/zero
student@okan-lab:/$ head -n 3 /proc/meminfo
MemTotal:        8222320 kB
MemFree:         7069620 kB
MemAvailable:    7539020 kB
student@okan-lab:/$ echo $$
18432
student@okan-lab:/$ head -n 7 /proc/$$/status
Name:   bash
Umask:  0002
State:  S (sleeping)
Tgid:   18432
Ngid:   0
Pid:    18432
```

```
PPid:   18377
```

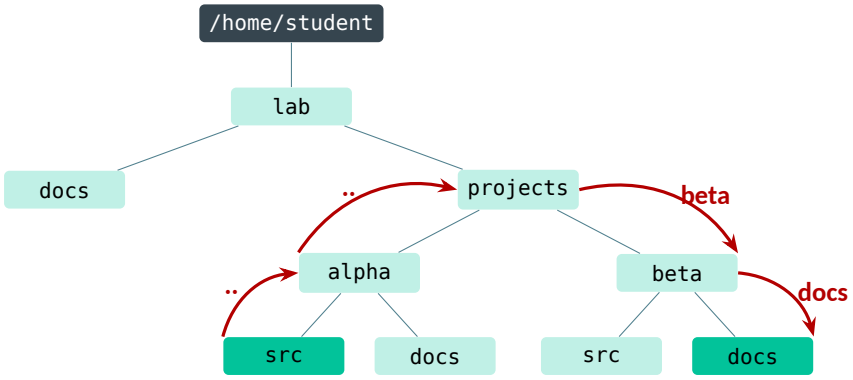
- `/dev` içindeki `c`, *karakter aygıtlarını* (character devices) işaretler. `ls -l` boyut yerine iki sürücü numarası (büyük, küçük; major, minor) gösterir, örneğin `1, 3`. `/dev/null` kendisine yazılan her şeyi atar (onu `strace` ile kullandık); `/dev/zero` sıfır baytları üretir; `/dev/tty` terminalinizdir. Programlar bu aygıtları sıradan dosyalarla aynı `open`/`read`/`write` sistem çağrılarıyla kullanır (2. hafta, G/Ç).
- `/proc` hiçbir diskte saklanmaz. Dosyalarından birini okuduğunuzda metni çekirdek anında yazar. `/proc/meminfo`, test makinemizde yaklaşık 8 GB RAM olduğunu gösteriyor ($8\,222\,320$ kB).
- Kabuk `$$` yerine kendi PID'sini koyar. `/proc/<PID>/status`, o sürecin PCB'sinin (3. hafta) bir kısmını gösterir: adı (`bash`), durumu S (sleeping, uyuyor — `head`'in bitmesini bekliyor), PID'si ve ebeveyninin PID'si (`PPid`). 13. hafta `/proc`'a ve `ps`'ye geri dönecek.

7 Dolaşmak: cd ve Yollar

7.1 Mutlak ve göreli yollar

7. haftadaki gibi bir dosya ya da dizin iki biçimde adlandırılabilir:

- **Mutlak yol** (absolute path) `/` ile başlar ve kökten aşağı yürür. Her yerden aynı anlama gelir: `/home/student/lab/projects/beta/docs`.
- **Göreli yol** (relative path) `/` ile başlamaz; geçerli dizinden yürür. `..` "bir düzey yukarı" demektir.



Şekil 5: `projects/alpha/src`'den `projects/beta/docs`'a: `../../beta/docs` (yukarı, yukarı, aşağı, aşağı).

7.2 Uygulamada cd

```
student@okan-lab:~$ cd ~/lab
student@okan-lab:~/lab$ pwd
/home/student/lab
student@okan-lab:~/lab$ cd projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ pwd
/home/student/lab/projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ cd ..
student@okan-lab:~/lab/projects/alpha$ pwd
/home/student/lab/projects/alpha
student@okan-lab:~/lab/projects/alpha$ cd ../../docs
```

```
student@okan-lab:~/lab/docs$ pwd
/home/student/lab/docs
student@okan-lab:~/lab/docs$ cd /home/student/lab/projects/beta
student@okan-lab:~/lab/projects/beta$ pwd
/home/student/lab/projects/beta
student@okan-lab:~/lab/projects/beta$ cd -
/home/student/lab/docs
student@okan-lab:~/lab/docs$ cd -
/home/student/lab/projects/beta
student@okan-lab:~/lab/projects/beta$ cd
student@okan-lab:~$ pwd
/home/student
```

Nelere dikkat etmeli:

- Her `cd`'den sonra istem yeni geçerli dizini gösterir.
- `cd` başarılı olduğunda *hiçbir şey* yazmaz ("haber yoksa iyi haber" bir UNIX kuralıdır). Tek istisna `cd -`'dir; geri döndüğü dizini yazar.
- `~/lab/projects/alpha`'dan `cd ../../docs` iki kez yukarı çıkar (`~/lab`'a), sonra `docs`'a iner.
- `/home/student/lab/projects/beta` mutlak yolu nerede olursanız olun çalışır.
- Argümansız `cd` sizi eve, `cd /` köke götürür.

7.3 Beş özel ad

Ad	Anlamı	Örnek	`~/lab/projects/alpha`'dan sonuç
`/`	ağacın kökü	`cd /`	`/`
`~`	ev dizininiz	`cd ~/lab`	`/home/student/lab`
`.`	geçerli dizin	`ls .`	`alpha`'yı listeler (`ls` ile aynı)
`..`	üst dizin	`cd ../beta`	`~/lab/projects/beta`
`-`	bir önceki dizin (yalnızca `cd`)	`cd -`	önceki yerinize döner

`.` ve `..` gerçek dizin girdileridir (`ls -a` onları gösterir); `~` ve `-` ise kabuğun kısayollarıdır: `~` yerine `$HOME` değişkeninin değeri konur, `cd -` ise `help cd`'nin açıkladığı gibi `$OLDPWD` değişkenini kullanır (Bölüm 8). `/`'nin `..` girdisi yine `/`'dir: kökün üstüne çıkamazsınız. `.` adı 11. ve 14. haftalarda önem kazanacak: `./script.sh` geçerli dizindeki bir betiği çalıştırır.

7.4 cd neden bir kabuk yerleşiği olmak zorunda?

```
student@okan-lab:~/lab$ /bin/pwd
/home/student/lab
student@okan-lab:~/lab$ strace -e trace=getcwd /bin/pwd
getcwd("/home/student/lab", 4096)       = 18
/home/student/lab
+++ exited with 0 +++
student@okan-lab:~/lab$ strace -e trace=chdir bash -c 'cd /tmp'
chdir("/tmp")                           = 0
+++ exited with 0 +++
```

```
student@okan-lab:~/lab$ pwd
/home/student/lab
```

- `/bin/pwd` ayrı bir programdır (yerleşik pwd değil); geçerli dizini çekirdeğe `getcwd` sistem çağrısıyla sorar. `/home/student/lab` yazar: çocuk süreç ebeveyninin geçerli dizinini devralır.
- `bash -c 'cd /tmp'`, `cd /tmp` komutunu çalıştıran bir *çocuk* kabuk başlatır. Çekirdeğin `chdir("/tmp")` çağrısı başarılı olur — ama sonrasında bizim kabuğumuz hâlâ `~/lab` içindedir.

Geçerli dizin sürece aittir

Geçerli çalışma dizini, çekirdeğin her süreç için PCB'sinin yanında tuttuğu durumun bir parçasıdır (3. hafta). Bir süreç yalnızca *kendi* geçerli dizinini değiştirebilir, ebeveyninkini asla. `cd /usr/bin` içinde bir program olsaydı kabuk bir çocuk oluşturur, çocuk *kendi* dizinini değiştirip sonlanırdı — kabuk ise olduğu yerde kalırdı. Bu nedenle `cd` kabuğun içine yerleşiktir: `bash`, `chdir()` çağrısını kendi üzerinde yapar. `type cd` bunu doğrular: "cd is a shell builtin". Aynı gerekçe `exit` ve kabuk değişkenlerini ayarlayan komutlar için de geçerlidir.

8 Yardım Almak

Kimse her seçeneği hatırlamaz. Nereye bakacağınızı bilin:

`komut --help` Seçeneklerin özeti; çoğu program yazar.

`man komut` Tam *kılavuz sayfası* (manual page); NAME, SYNOPSIS, DESCRIPTION, OPTIONS ve EXAMPLES gibi bölümlere ayrılır. Ok tuşları ya da Boşluk (sonraki sayfa) ile ilerleyin, `/sözcük` ve Enter ile arayın (sonraki eşleşme için n), q ile çıkın. `man man`, `man`'in kendisini açıklar.

`help komut` Kabuk yerleşikleri (`cd`, `pwd`, `history`) için yardım; bunların program dosyası ve çoğu zaman kendi kılavuz sayfası yoktur.

`type komut` Bir komutun yerleşik mi, takma ad mı, yoksa (nerede duran) bir program dosyası mı olduğunu söyler.

`whatis` / `apropos` `whatis ls`, kılavuzdaki tek satırlık açıklamayı yazar; `apropos directory` bütün kılavuz açıklamalarında bir anahtar sözcük arar — komutun adını bilmediğinizde işe yarar.

`tldr komut` (isteğe bağlı, `sudo apt install tldr`) yaygın kullanımlar için topluluğun yazdığı kısa örnekler.

```
student@okan-lab:~$ ls --help | head -n 12
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
  -a, --all                  do not ignore entries starting with .
  -A, --almost-all           do not list implied . and ..
```

```
student@okan-lab:~$ type cd pwd ls
cd is a shell builtin
pwd is a shell builtin
ls is /usr/bin/ls
student@okan-lab:~$ help cd | head -n 5
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.

    Change the current directory to DIR.  The default DIR is the value of the
```

Bir SYNOPSIS satırını okumak: `ls [OPTION]... [FILE]...` “herhangi sayıda seçenek, ardından herhangi sayıda dosya, hepsi isteğe bağlı” demektir — köşeli parantezler isteğe bağlı kısımları işaretler, `...` ise “tekrarlanabilir” anlamına gelir. `ls --help` çıktısı | `head -n 12` ile kısaltıldı (yalnızca ilk 12 satır; borular 12. haftanın konusu). Türkçe yerel ayarlı bir sistemde `--help` çıktısı da büyük olasılıkla Türkçe olacaktır.

Test makinemiz kılavuz sayfaları olmayan *küçültülmüş* (minimised) bir Ubuntu olduğundan bu notlar `man` çıktısı yerine `--help` çıktısı gösteriyor; normal kurulumlarda, WSL’de ve laboratuvar bilgisayarlarında kılavuzlar vardır (küçültülmüş bir sistemde `sudo unminimize` onları geri yükler).

9 Akıllı Çalışma: Tamamlama, Geçmiş, Kısayollar

9.1 Sekme tamamlama

Bir komutun, dosyanın ya da dizinin adının ilk harflerinden sonra Tab’a basın; kabuk adı tamamlar (tab completion). Aşağıdaki oturumda kullanıcı `cd pro` Tab `al` Tab yazdı ve kabuk bunu `cd projects/alpha/` olarak tamamladı; `ls ~/lab/docs/sy` Tab ise `ls ~/lab/docs/syllabus.txt` oldu (tamamlanan adın sonuna bir boşluk eklenerek). Başlangıç birden çok ada uyuyorsa (`notes_0` üç dosya olabilir) hiçbir şey olmuyor gibi görünür: adayların listesini görmek için Tab’a *iki kez* basın. Sekme tamamlama hem yazmaktan kurtarır *hem de* yazım hatalarını önler.

9.2 Komut geçmişi

```
student@okan-lab:~$ cd ~/lab
student@okan-lab:~/lab$ cd projects/alpha/
student@okan-lab:~/lab/projects/alpha$ pwd
/home/student/lab/projects/alpha
student@okan-lab:~/lab/projects/alpha$ cd
student@okan-lab:~$ ls ~/lab/docs/syllabus.txt
/home/student/lab/docs/syllabus.txt
student@okan-lab:~$ whoami
student
student@okan-lab:~$ !!
whoami
student
student@okan-lab:~$ history
    1  cd ~/lab
    2  cd projects/alpha/
    3  pwd
    4  cd
```

```
    5  ls ~/lab/docs/syllabus.txt
    6  whoami
    7  whoami
    8  history
student@okan-lab:~$ !5
ls ~/lab/docs/syllabus.txt
/home/student/lab/docs/syllabus.txt
```

Kabuk komutlarınızı hatırlar (history). `history` onları numaralarıyla yazar; `!!` son komutu, `!5` ise 5 numaralı komutu tekrarlar (kabuk önce çalıştıracağı komutu yazar). ↑/↓ ok tuşları geçmişte gezinir, `Ctrl+R` ise siz yazdıkça geçmişte geriye doğru arar. Geçmiş, kabuk kapanırken `~/.bash_history` dosyasına kaydedilir.

9.3 Klavye kısayolları

Tuş	Etkisi
`Tab` / `Tab Tab`	bir adı tamamla / adayları listele
`↑` / `↓`	geçmişteki önceki / sonraki komut
`Ctrl+R`	geçmişte geriye doğru ara
`Ctrl+C`	çalışan komutu kes (durdur) — SIGINT sinyalini gönderir (13. hafta)
`Ctrl+L`	ekranı temizle (`clear` gibi)
`Ctrl+A` / `Ctrl+E`	satırın başına / sonuna atla
`Ctrl+D`	girdinin sonu; boş istemde kabuğu kapatır (`exit` gibi)

Çoğu terminal penceresinde kopyalama ve yapıştırma `Ctrl+Shift+C` ve `Ctrl+Shift+V` ile yapılır, çünkü `Ctrl+C` zaten “kes” anlamına gelir.

10 Komut Başvurusu: 9. Hafta

Komut	Amaç
`whoami`	kullanıcı adınızı yazar
`hostname`	bilgisayarın adını yazar
`date`	tarihi ve saati yazar
`echo metin`	argümanlarını yazar (kabuk onları genişlettikten sonra)
`uname [-r|-a]`	çekirdek adı / sürümü / bütün bilgiler
`clear`	ekranı temizler (`Ctrl+L`)
`pwd`	geçerli çalışma dizinini yazar (mutlak yol)
`ls [-laAhRFdStr1i] [dizin...]`	dizin içeriğini listeler (Bölüm 5)
`tree [dizin]`	dizin ağacını çizer
`cd [dizin]`	dizin değiştirir; `~ . .. - /`; argümansız: eve
`cat dosya`	küçük bir dosyayı yazar (10. hafta `cat`, `less`, `head`, `tail`'i işler)
`head -n N dosya`	bir dosyanın ilk N satırını yazar
`type komut`	yerleşik mi, takma ad mı, program dosyası mı?
`komut --help`	bir programın kısa yardımı
`man komut`	kılavuz sayfası (q çıkar, `/sözcük` arar)
`help komut`	bir kabuk yerleşiğinin yardımı
`whatis komut / apropos sözcük`	tek satırlık açıklama / kılavuzlarda arama
`history, !!, !n`	önceki komutları listeler / tekrarlar
`strace komut`	bir komutun sistem çağrılarını gösterir (2. hafta)
`bash dosya.sh`	bir kabuk betiğini çalıştırır (örneğin lab kurulum betiği)

11 Sık Görülen Hatalar ve Anlamları

Hata mesajları kesindir; okuyun. Aşağıdakilerin hepsi `~/lab` içinde üretildi:

```
student@okan-lab:~/lab$ LS
bash: LS: command not found
student@okan-lab:~/lab$ sl
bash: sl: command not found
student@okan-lab:~/lab$ ls -z
ls: invalid option -- 'z'
Try 'ls --help' for more information.
student@okan-lab:~/lab$ ls Docs
ls: cannot access 'Docs': No such file or directory
student@okan-lab:~/lab$ cd nowhere
bash: cd: nowhere: No such file or directory
student@okan-lab:~/lab$ cd README.txt
bash: cd: README.txt: Not a directory
student@okan-lab:~/lab$ ls docs/final report.txt
ls: cannot access 'docs/final': No such file or directory
ls: cannot access 'report.txt': No such file or directory
student@okan-lab:~/lab$ ls "docs/final report.txt"
'docs/final report.txt'
student@okan-lab:~/lab$ ls docs/final\ report.txt
'docs/final report.txt'
```

```
student@okan-lab:~/lab$ cd docs/final report.txt
bash: cd: too many arguments
student@okan-lab:~/lab$ cd /root
bash: cd: /root: Permission denied
student@okan-lab:~/lab$ pwd
/home/student/lab
```

`command not found` Bir yazım hatası, yanlış harf büyüklüğü (LS) ya da kurulu olmayan bir program (`sl`). Normal bir Ubuntu'da kabuk ayrıca bir paket önerir; örneğin "Command 'sl' not found, but can be installed with: sudo apt install sl".

`invalid option` Komutun böyle bir seçeneği yok. Mesajın kendisi `ls --help`'i öneriyor.

`No such file or directory` Ad yanlış (harf büyüklüğünü unutmayın: `Docs` ≠ `docs`) ya da sandığınız dizinde değilsiniz. `pwd` ve `ls` çalıştırın, sekme tamamlamayı kullanın. (Türkçe sistemde: `Böyle bir dosya ya da dizin yok`.)

`Not a directory` `cd` bir dizin ister; `README.txt` ise bir dosyadır.

Tek ad için iki hata Kabuk komut satırını boşluklardan böler; bu yüzden `docs/final report.txt` *iki* argüman olur: `docs/final` ve `report.txt`. Adı tırnak içine alın (`"docs/final report.txt"`) ya da boşluğu ters eğik çizgiyle kaçırın (`docs/final\ report.txt`). `cd` ile iki argüman `too many arguments` hatası verir.

`Permission denied` İçeri girmenize izin yok (`/root` yöneticiye aittir). Bu, koruma sisteminin tasarlandığı gibi çalışmasıdır (11. hafta). Buna komutların başına `sudo` koyarak karşılık *vermeyin*.

`$'\r': command not found` Windows satır sonlarına sahip bir betik (Bölüm 2.4).

> **sudo hakkında bir uyarı**
>
> `sudo`, *sizin* parolanızı sorduktan sonra tek bir komutu yönetici olan `root` olarak çalıştırır. Bu derste ona yalnızca yazılım kurmak için ihtiyacınız var (`sudo apt install tree`). `root` olarak çalışırken 2. ve 11. haftaların koruma mekanizmaları sizi kendi hatalarınızdan artık korumaz: tek bir yanlış komut sisteme zarar verebilir. Bir komut başarısız oldu diye asla `sudo` kullanmayın.

12 Özet

Yedi cümlede 9. hafta

1. Bir Linux kabuğuna laboratuvar bilgisayarlarında, Windows'ta WSL ile (`wsl --install`) ya da bir VirtualBox sanal makinesinde ulaşılır; `mcmp117_lab_setup.sh` betiği alıştırma klasörü `~/lab`'ı oluşturur.
2. Terminal metni gösterir, kabuk (`bash`) komutları kullanıcı modunda yorumlar, çekirdek ise işi sistem çağrılarıyla yapar.
3. Harici bir komut için kabuk fork yapar, çocuk program dosyasını exec eder ve kabuk bekler (3. hafta); `cd` gibi yerleşikler kabuğun içinde çalışır.
4. Bir komut satırı *komut -seçenekler argümanlar* biçimindedir; sözcükler boşluklarla ayrılır ve harf

büyüklüğü önemlidir.

5. `pwd` nerede olduğunuzu, `ls` (`-l -a -h -R -F` ile) orada ne olduğunu gösterir; `ls -l` 7. haftanın düğüm alanlarını sergiler.
6. Linux'ta `/`'den başlayan tek bir ağaç vardır; `cd` mutlak ya da göreli yollarla ve `~ . .. -` adlarıyla dolaşır.
7. `--help`, `man`, `help` ve `type` komutları açıklar; Tab, geçmiş ve `Ctrl+C`/`Ctrl+L` zaman kazandırır.

13 Sözlük

Mutlak yol (absolute path) Kök `/` ile başlayan yol; her dizinden geçerlidir.

Yerleşik komut (builtin) Kabuğun yeni bir süreç oluşturmadan kendisinin yürüttüğü komut (`cd`, `pwd`, `echo`, `history`).

Geçerli çalışma dizini (current working directory) Bir sürecin göreli yolları çözdüğü dizin; her sürecin durumunun parçasıdır.

FHS Dosya Sistemi Hiyerarşi Standardı (Filesystem Hierarchy Standard): hangi üst düzey dizinin neyi tuttuğu.

Gizli dosya (hidden file) Adı nokta ile başlayan dosya; yalnızca `ls -a` gösterir.

Ev dizini (home directory) Bir kullanıcının kendi dizini, `/home/<kullanıcı>`; kısaltması `~`.

Seçenek (option) Bir komutun nasıl çalışacağını değiştiren, `-` ya da `--` ile başlayan sözcük.

İstem (prompt) Kabuğun bir komuta hazır olduğunda yazdığı metin.

Göreli yol (relative path) Geçerli dizinden başlayan yol; `..` kullanabilir.

Kabuk (shell) Komut yorumlayıcısı: komutları okuyup çalıştıran, kullanıcı modunda bir sistem programı.

Terminal (öykünücü; terminal emulator) Tuş vuruşlarını kabuğa ileten ve onun çıktısını gösteren pencere programı.

WSL Linux için Windows Alt Sistemi (Windows Subsystem for Linux): gerçek bir Linux çekirdeğiyle bir Linux dağıtımını Windows içinde çalıştırır.

14 Laboratuvar Föyü

Önce lab klasörünü yeniden oluşturun (`bash mcmp117_lab_setup.sh`); aksi belirtilmedikçe her alıştırmaya `~/lab` içinde başlayın. Kullandığınız *komutları* ve yanıtları yazın. Toplam: 100 puan. Çözümler Bölüm 16'de — önce kendiniz deneyin! Bu laboratuvarda küçük bir dosyayı yazdırmak için `cat dosya`, ilk N satırını yazdırmak için `head -n N dosya` kullanabilirsiniz.

A Bölümü --- Başlangıç (13 puan)

A1 (4 puan) `whoami`, `hostname`, `date` ve `uname -r` komutlarını çalıştırın. Çıktıları yazın ve her komutun neyi bildirdiğini birer cümleyle söyleyin.

A2 **(4 puan)** Lab kurulum betiğini çalıştırın. Lab klasörünüzün *mutlak* yolu nedir? Bunu hangi komut kanıtlar?

A3 **(5 puan)** Her komut satırı için komutu, seçenekleri ve argümanları belirtin: (a) `ls -l -a docs`, (b) `ls -lh /var/log /tmp`, (c) `head -n 3 data/cities.txt`, (d) `cd`, (e) `ls --all`.

B Bölümü --- Etrafa bakmak (30 puan)

B4 **(6 puan)** `ls`, `~/lab` içinde kaç girdi gösteriyor? Bunların kaçı dizin? Hangi seçenek yanıtı apaçık hâle getirir?

B5 **(6 puan)** `docs` gizli bir dosya içeriyor. Adını bulun ve içeriğini yazdırın.

B6 **(6 puan)** `docs` içindeki en büyük dosya hangisi ve kaç bayt? `ls`'nin boyuta göre sıralayan seçeneğini kullanın (`ls --help` içinde bulun).

B7 **(6 puan)** `logs/access.log` dosyasının bayt cinsinden tam boyutunu ve `ls -lh`'nin gösterdiği boyutu verin. İkisinin neden uyuştuğunu hesapla gösterin.

B8 **(6 puan)** `photos` içinde kaç dosya var? Kaçı tam olarak `.jpg` (küçük harf) ile bitiyor? Hangileri bitmiyor?

C Bölümü --- Dolaşmak (24 puan)

C9 **(6 puan)** `~/lab`'dan *tek* bir göreli `cd` ile `projects/beta/docs`'a gidin ve geçerli dizini yazdırın.

C10 **(6 puan)** Oradan tek bir göreli `cd` ile `projects/alpha/src`'ye gidin. Komutu yazın ve `pwd` ile kontrol edin.

C11 **(6 puan)** `~/lab/projects` içinden başlayarak sırasıyla `cd alpha`, `cd ../beta/src`, `cd ..`, `cd -` yazıyorsunuz. Her komuttan sonraki dizini *tahmin edin*, sonra kontrol edin.

C12 **(6 puan)** *Herhangi bir dizinden* sizi `~/lab/src`'ye götüren iki farklı komut yazın. `cd lab/src` neden yalnızca bazı dizinlerde çalışır?

D Bölümü --- Sistem ve yardım (18 puan)

D13 **(6 puan)** `ls` programı nerede saklanıyor? Ubuntu 24.04'te `/bin` tam olarak nedir? Dağıtımın adını ve sürümünü hangi dosya söyler ve sizin sisteminizde ne yazıyor?

D14 **(6 puan)** Kabuğunuzun PID'sini ve `/proc/<PID>/status` dosyasının `Name`, `State`, `Pid` ve `PPid` satırlarını yazdırın. Bu, 3. haftanın hangi veri yapısına bir bakıştır ve durum neden `S`'dir?

D15 **(6 puan)** `ls --help` ile her satırda bir ad listeleyen seçeneği ve sıralamayı ters çeviren seçeneği bulun. İkisini birlikte `docs` üzerinde kullanın. Sonra `help cd` ile `cd -`'nin ne yaptığını açıklayın.

E Bölümü --- Zorlu sorular (15 puan)

E16 **(5 puan) Hazine avı.** `~/lab`'da başlayın ve yalnızca `cd`, `ls`, `pwd` ve `cat` kullanın. (1) `README.txt`'yi okuyun: "a small C program" tutan dizine gidin. (2) O program ne yazar? (3) Tek bir göreli `cd` ile kısa metin belgelerinin dizinine geçin. (4) Oradaki bir dosyanın adında boşluk var; son satırı paroladır. (5) Tek bir göreli `cd` ile `projects/beta/src`'de bitirin ve `pwd` çalıştırın.

E17 (5 puan) Bağlantı sayıları. `ls -ld`, `~/lab` için 10, `projects` ve `projects/alpha` için 4, `docs` için 2 bağlantı sayısı gösteriyor. Dört sayıyı da 7. haftadaki sabit bağlantı fikriyle açıklayın ve `projects/beta`'nın bağlantı sayısını tahmin edin.

E18 (5 puan) Neden yerleşik? `type cd`, `strace -e trace =chdir bash -c 'cd /tmp'` ve ardından `pwd` çalıştırın. 3. haftanın süreç kavramını kullanarak `cd`'nin neden sıradan bir program olamayacağını açıklayın.

15 Tekrar Soruları

Kavramsal

1. Terminal, kabuk ve çekirdek arasındaki farkı açıklayın. Bunlardan hangileri kullanıcı modunda çalışır?
2. `ls docs` yazdığınızda kabuğun attığı adımları sıralayın. 3. haftanın hangi sistem çağrıları işin içindedir?
3. Yerleşik bir komutla harici bir komut arasındaki fark nedir? Bir komutun hangisi olduğunu nasıl anlarsınız?
4. Gizli dosya nedir? Gizli dosyaları nasıl listelersiniz?
5. `drwxrwxr-x 4 student student 4096 Oct 12 08:49 projects` satırının her sütununu açıklayın.
6. Linux'un sürücü harflerine neden ihtiyacı yoktur? `/proc` nedir ve neden sanal dosya sistemi olarak adlandırılır?
7. Mutlak ve göreli yol arasındaki farkı, aynı dizini adlandıran birer örnekle açıklayın.
8. `~`, `.`, `..` ve `-` `cd` için ne anlama gelir? Bunlardan hangileri gerçek dizin girdileridir?
9. `ls docs/final report.txt` neden iki hata mesajı yazar? Komutu düzeltmenin iki yolunu verin.

Sayısal

10. Bir dosya $3\,500\,000$ bayttır. `ls -lh` ne gösterir?
11. Bir dosya $10\,300$ bayttır. `ls -lh` ne gösterir?
12. Bir dizinde 5 alt dizin ve 12 dosya var. Bağlantı sayısı kaçtır?
13. Bir dizinde her biri 4 KiB'tan küçük 7 dosya var ve dosya sistemi 4 KiB'lık bloklar kullanıyor. `ls -l` çıktısının `total` satırı ne gösterir?
14. `/home/student/lab/projects/alpha/docs` içindesiniz. `/home/student/lab/data`'ya giden en kısa göreli yolu yazın. Kaç düzey yukarı çıkar?

Sayısal soruların yanıtları

10. $3\,500\,000 / 2^{20} = 3\,500\,000 / 1\,048\,576 \approx 3.338$; bir ondalık basamak, yukarı yuvarlanır: `3.4M`.
11. $10\,300 / 1024 \approx 10.06$; 10 ve üzerinde ondalık gösterilmez ve değer yukarı yuvarlanır: `11K`.
12. $2 + 5 = 7$ (dosyalar sayılmaz).

13. Her dosya bir blok kaplar: $7 \times 4 = 28$, yani `total 28`.
14. `../../../data`: üç düzey yukarı (`docs`'un üstü `alpha`'ya, sonra `projects`'e, sonra `lab`'a), sonra `data`'ya iner.

16 Laboratuvar Föyü Çözümleri

Aşağıdaki çıktılar `code/out_lab.txt` (betik: `code/demo_lab.sh`) ve bu haftanın diğer kayıtlarından alınmıştır. Tarihler, PID'ler ve sistem ayrıntıları sizin bilgisayarınızda farklı olacaktır.

A1. Bkz. Bölüm 4.3: `student`, `okan-lab`, o anki tarih ve saat ve çekirdek sürümü (makinemizde `6.18.44-fc-v114`). `whoami`: kullanıcı adı; `hostname`: bilgisayar adı; `date`: tarih ve saat; `uname -r`: Linux çekirdeğinin sürümü.

A2. Betik `Lab folder ready: /home/student/lab` yazar (Bölüm 2.4). `cd ~/lab` ve ardından `pwd`, mutlak yolu `/home/student/lab` olarak yazar (sizin kullanıcı adınızla).

A3. (a) komut `ls`, seçenekler `-l -a`, argüman `docs`; (b) `ls`, seçenekler `-l -h` (birleşik), argümanlar `/var/log` ve `/tmp`; (c) `head`, 3 değerli `-n` seçeneği, argüman `data/cities.txt`; (d) `cd`, seçenek yok, argüman yok (eve gider); (e) `ls`, uzun seçenek `--all`, argüman yok.

B4. 9 girdi: 8 dizin ve `README.txt` dosyası. `ls -F` dizinleri `/` ile işaretler (`ls -l` `d` gösterir).

B5. `ls -a docs` `.settings` dosyasını ortaya çıkarır; `cat docs/.settings` `editor=nano` ve `color=on` yazar.

B6. `-S` boyuta göre, en büyük önce sıralar: `ls -lS docs` en üste `syllabus.txt`'yi (578 bayt), ardından `linux_quotes.txt`'yi (302) ve `final report.txt`'yi (53) koyar.

B7. `ls -l logs` $23\,604$ bayt verir; `ls -lh` `24K` gösterir, çünkü $23\,604/1024 \approx 23.05$ ve `ls -h` yukarı yuvarlar.

B8. 15 dosya. 12'si `.jpg` ile biter (`img001.jpg` … `img012.jpg`); diğer üçü `IMG_014.JPG` (büyük harf — Linux'ta farklı bir ad), `holiday.jpeg` ve `img013.png`'dir. (Gelecek hafta `*.jpg` gibi joker karakterler onları sizin için sayacak.) `ls photos` 15 adın hepsini üç sütunda gösterir.

C9 ve C10. `cd projects/beta/docs`, sonra `cd ../../alpha/src` (beta'ya çık, `projects`'e çık, `alpha/src`'ye in); `pwd` önce `/home/student/lab/projects/beta/docs`, sonra `/home/student/lab/projects/alpha/src` yazar.

C11. `cd alpha` sonrasında: `~/lab/projects/alpha`; `cd ../beta/src` sonrasında: `~/lab/projects/beta/src`; `cd ..` sonrasında: `~/lab/projects/beta`; `cd -` ise `~/lab/projects/beta/src`'ye döner ve bu dizini yazar.

C12. `cd /home/student/lab/src` (mutlak; kendi kullanıcı adınızı yazın) ve `cd ~/lab/src` (kabuk `~`'yi ev dizininizin mutlak yoluna genişletir). `cd lab/src` görelidir: yalnızca geçerli dizin `lab`'ı içeren `/home/student` olduğunda çalışır.

D13. `type ls` → `/usr/bin/ls`; `ls -ld /bin` → `usr/bin`'e bir sembolik bağlantı; `/etc/os-release` dağıtımın adını verir; makinemizde `PRETTY_NAME="Ubuntu 24.04.4 LTS"` (Bölüm 6).

D14. `echo $$`, ardından `head -n 7 /proc/$$/status` (Bölüm 6; PID her seferinde farklıdır). Bu, kabuğun *süreç denetim bloğuna* (PCB, 3. hafta) bir bakıştır: ad, durum, PID, ebeveyn PID. Durum S'dir (uyuyor), çünkü head çalışırken kabuk, çocuğunu beklemek için `wait()` içinde engellenmiştir.

D15. `-1` (her satırda bir) ve `-r` (ters):

```
student@okan-lab:~/lab$ ls -1 -r docs
syllabus.txt
notes_03.txt
notes_02.txt
notes_01.txt
linux_quotes.txt
'final report.txt'
```

`help cd` şunu söyler: *If DIR is "-", it is converted to $OLDPWD*; yani `cd -` bir önceki dizine gider (Bölüm 8).

E16 (hazine avı). (1) `README.txt`, `src/` dizininin "a small C program" tuttuğunu söyler: `cd src`. (2) `cat hello.c`: program `Hello from the lab!` yazar. (3) `cd ../docs`. (4) `cat "final report.txt"`: parola `This file name contains a space.` (5) `cd ../projects/beta/src`; `pwd` `/home/student/lab/projects/beta/src` yazar.

E17. Bir dizinin bağlantı sayısı $= 2 + s$ (s = alt dizin sayısı): üst dizindeki adı, kendi `.` girdisi ve her alt dizindeki bir `..`. `~/lab`: $s = 8$, $2 + 8 = 10$; `projects`: `alpha`, `beta`, $2 + 2 = 4$; `alpha`: `docs`, `src`, 4; `docs`: 2. `projects/beta` için tahmin: 4.

```
student@okan-lab:~/lab$ ls -ld ~/lab ~/lab/projects ~/lab/projects/alpha ~/lab/docs
drwxrwxr-x 10 student student 4096 Oct 12 08:49 /home/student/lab
drwxrwxr-x  2 student student 4096 Oct 12 08:49 /home/student/lab/docs
drwxrwxr-x  4 student student 4096 Oct 12 08:49 /home/student/lab/projects
drwxrwxr-x  4 student student 4096 Oct 12 08:49 /home/student/lab/projects/alpha
```

E18. Bkz. Bölüm 7.4: `type cd` "cd is a shell builtin" yazar; çocuk `bash` `chdir("/tmp")` çağrısını başarıyla yapar, ama ardından `pwd` hâlâ `/home/student/lab` yazar. Geçerli dizin sürece özgü bir durumdur; bir çocuk ebeveyninin dizinini değiştiremez, dolayısıyla `cd` kabuk sürecinin kendi içinde çalışmak zorundadır.

Puanlama ipuçları. Tarihler, PID'ler ve sistem ayrıntıları farklı olsa da komutlar doğruysa tam puan verin. C9–C12 ve E16'da göreli yol istenen yerde mutlak yol kullanılmışsa puan kırın.

17 İleriye Bakış: 10. Hafta

Gelecek hafta yalnızca bakmayı bırakıp `~/lab` içindeki dosya sistemini *değiştirmeye* başlıyoruz:

- oluşturmak ve silmek: `mkdir -p`, `touch`, `rm`, `rmdir` — ve `rm`'nin neden geri dönüşüm kutusu olmadığı;
- kopyalamak ve taşımak: `cp -r -i -v`, `mv`;
- joker karakterler (wildcards) `*`, `?`, `[abc]` ve küme parantezi genişletmesi (brace expansion) `{a,b}`, `{1..5}` — komut değil kabuk tarafından genişletilir (bu haftaki `echo` ve `~` örneğini hatırlayın);
- dosyaları görüntülemek: `cat`, `less`, `head`, `tail`, `wc`, `file`, `stat`, `du`, `df`;
- `ln` ve `ln -s` ile sabit ve sembolik bağlantılar (7. haftadaki düğümler).

Dersten önce kendi bilgisayarınızda çalışan bir terminal ve lab klasörü bulunduğundan emin olun.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 2.2 (User and Operating-System Interface: komut yorumlayıcıları) ve Bölüm 13.3 (Directory Structure).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - 5. bölüm, "Interlude: Process API" (bir kabuğun `fork`, `exec`, `wait`'i nasıl kullandığı): `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
  - 39. bölüm, "Interlude: Files and Directories": `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`
- W. Shotts, *The Linux Command Line* (ücretsiz kitap), 1. kısım, 1–5. bölümler: `https://linuxcommand.org/tlcl.php`
- Microsoft, "WSL ile Windows'a Linux yükleme": `https://learn.microsoft.com/tr-tr/windows/wsl/install`
- Filesystem Hierarchy Standard 3.0: `https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.html`